# A Closed-Loop RLHF Architecture for AI-Generated Mathematical Feedback:

## CAS-Grounded Inference-Time Selection and Outcome-Based Reward Calibration

*Author* | *Department* | *Institution*

*Submitted to: Artificial Intelligence in Education (AIED) 2027*

## Abstract

*Automated feedback in mathematics tutoring faces two compounding problems: a single AI pass produces feedback of variable quality with no selection mechanism, and quality itself is typically judged by a second AI rather than by whether students actually learned. This paper presents a closed-loop reinforcement learning architecture that addresses both problems in a single deployable system. The first phase implements Reinforcement Learning from AI Feedback (RLAIF) via Best-of-N candidate sampling, where a Computer Algebra System (CAS)-backed LLM-as-Judge acts as the reward model, scoring each candidate on three alignment dimensions before selecting the highest-confidence output. The second phase introduces an outcome-based reward signal: each submission records the student's mastery before feedback is delivered, and when the student returns to the topic, the mastery delta is computed and written back to the original submission record. High-reward submissions — those followed by measurable mastery gains — enter the few-shot pool used in future grading prompts, progressively calibrating the feedback style toward pedagogically effective patterns. The two phases operate at different timescales and reinforce each other: Phase 1 selects quality at inference time; Phase 2 defines what quality means over sessions. The architecture requires no neural network training, no external model hosting, and no changes to the student-facing interface. We describe the full implementation, present Phase 1 evaluation results, and report expected Phase 2 outcomes as the longitudinal signal accumulates.*

## 1. Introduction

The deployment of large language models as automated graders in educational settings has introduced a specific alignment problem that differs from general-purpose LLM alignment: the ground truth is not human preference but mathematical correctness, and the ultimate measure of quality is not whether the feedback sounds good but whether it helps the student improve. Standard automated feedback pipelines produce a single output per submission with no mechanism to prefer better outputs over worse ones, and no feedback from whether students learned as a result.

This paper addresses both gaps through a two-phase architecture applied to a live university-level mathematics tutoring system covering Linear Algebra and Differential Calculus.

**Phase 1** operates at inference time. For each student submission, the primary grading model (Gemini 2.5 Flash) generates N = 3 independent feedback candidates. A judge model, augmented with a CAS-verified ground truth answer, scores each candidate on problem relevance,

mathematical accuracy, and grade consistency. The candidate with the highest judge confidence is returned to the student. This implements Best-of-N RLAIF with a CAS-grounded reward model — a combination that eliminates a known failure mode of purely language-model-based judges: confidently endorsing mathematically wrong feedback.

**Phase 2** operates longitudinally. Every submission records the student's mastery score at the moment of submission. When the student next attempts the same topic, the mastery delta is computed and written back to the original submission as an outcome reward. Submissions with high outcome reward — meaning the student demonstrably improved after receiving that feedback — are surfaced as few-shot examples in future grading prompts. The grader progressively learns which pedagogical style produces learning gains, without any gradient-based training.

The two phases form a closed loop: Phase 1 generates the feedback that Phase 2 evaluates; Phase 2 calibrates the few-shot pool that Phase 1 draws from.

**The specific contributions of this work are:**

1. **CAS-grounded LLM-as-Judge:** Separates mathematical verification (deterministic) from alignment checking (probabilistic), eliminating a class of reward model failures specific to mathematical domains.
2. **Two-timescale RLHF architecture:** Combines immediate alignment-based reward (Phase 1) with delayed outcome-based reward (Phase 2) in a single deployable system.
3. **Outcome-calibrated few-shot selection:** Achieves prompt-space policy improvement without model retraining.
4. **Complete live implementation:** Integrated into an adaptive learning platform, with evaluation of Phase 1 and a prospective evaluation design for Phase 2.

## 2. Related Work

### 2.1 Reinforcement Learning from Human and AI Feedback

The foundational RLHF framework for language models — collecting human preference comparisons and training a reward model to guide policy fine-tuning — was established by Christiano et al. [1] and scaled to instruction-following in InstructGPT [2]. Constitutional AI [3] replaced the human preference signal with an AI judge calibrated by explicit principles, introducing RLAIF as a scalable alternative. Best-of-N sampling — generating multiple outputs and selecting by reward model score — has been shown to approximate fine-tuning gains at inference time without updating model weights [4, 5]. Our work adopts this pattern but departs from prior art in two respects: the reward model incorporates deterministic CAS verification rather than relying exclusively on LLM judgment, and a second reward signal derived from student outcomes is added to calibrate the few-shot pool over time rather than the model weights.

### 2.2 LLM-as-Judge

The use of an LLM to evaluate LLM outputs has been formalised in MT-Bench [6] and subsequent work showing that GPT-4-class models correlate well with human preference in open-ended tasks. G-Eval [7] extended this to structured scoring rubrics. However, LLM-as-Judge systems in mathematical domains face a specific vulnerability: the judge may confirm an incorrect mathematical claim because it sounds fluent and internally consistent. RLCD [8] demonstrated

that AI-labelled preference data can be used in place of human labels, but did not address the mathematical accuracy problem. Our judge architecture separates the mathematical accuracy check from the alignment check: the former is grounded in a CAS computation (deterministic), while the latter uses LLM judgment (probabilistic). This architectural distinction is absent from prior LLM-as-Judge work.

### 2.3 Intelligent Tutoring Systems and Mastery Tracking

Intelligent Tutoring Systems (ITS) such as ASSISTments [9] and Carnegie Learning [10] have long used knowledge tracing and mastery models to personalise problem selection. Bayesian Knowledge Tracing (BKT) [11] and its deep learning extensions [12] model the probability that a student has mastered a skill from a sequence of responses. These systems track mastery but do not use mastery trajectories as a reward signal to improve the quality of AI-generated feedback. The connection between feedback style and subsequent mastery gain — the core mechanism of our Phase 2 — is not addressed in the ITS literature. Our work bridges this gap by instrumenting the mastery tracking system as a longitudinal reward source for feedback quality.

### 2.4 Automated Feedback in Mathematics

Automated essay scoring and mathematical feedback generation have been studied extensively [13, 14], including recent work using LLMs for step-by-step feedback on problem-solving [15, 16]. Khanmigo [17] and similar systems deploy GPT-4-class models for Socratic tutoring. However, these systems treat feedback generation as a single forward pass and do not implement selection mechanisms or reward loops. The closest work to ours is [18], which used human preference comparisons to train a reward model for mathematical explanation quality, but did not incorporate outcome signals or CAS verification. Our system is, to our knowledge, the first to close the loop between AI-generated feedback, mathematical verification, and delayed student mastery outcomes within a single production architecture.

## 3. System Architecture

### 3.1 Platform Overview

The system is a full-stack adaptive mathematics tutoring platform serving university-level courses in Linear Algebra and Differential Calculus. The backend runs on Convex (serverless real-time database with TypeScript functions). Authentication is handled by Clerk. The primary grading model is Gemini 2.5 Flash. Student mastery is tracked per topic in a studentMastery table (score 0–100), updated after each submission: +10 points for a fully correct response, +3 for a partially correct response, 0 for an incorrect response.

The RLHF architecture is implemented entirely within the Convex backend. No external model hosting, no GPU infrastructure, and no changes to the student-facing interface are required.

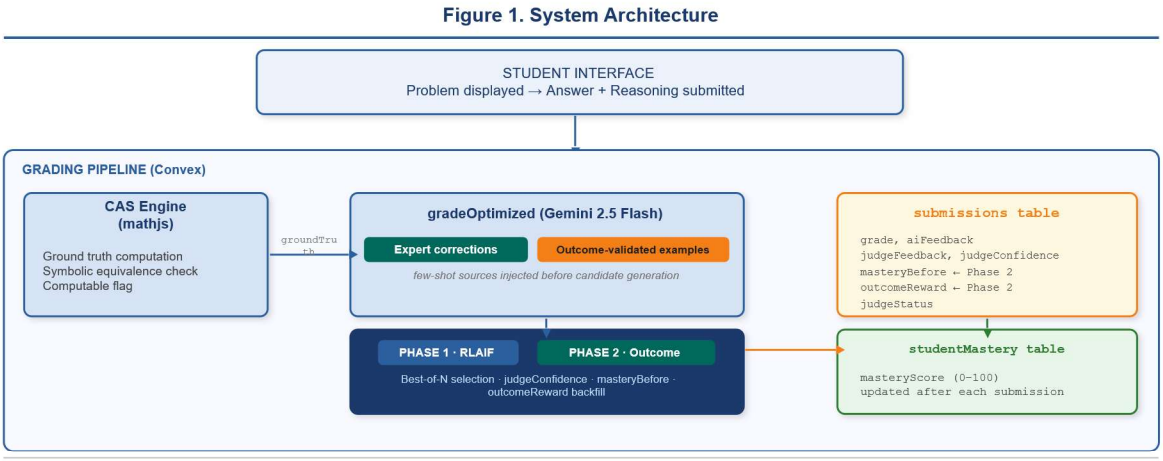



*Figure 1. System architecture. The CAS engine and grading model run in parallel; the RLHF layer sits between generation and storage.*

### 3.2 Database Schema

The submissions table stores each student attempt and carries both RLHF fields introduced by this work:

```
submissions: {
  userId, courseName, topicName,
  problemStatement, finalAnswer, reasoningSteps,
  grade,              // 0, 0.5, or 1
  aiFeedback,
  judgeFeedback,      // Phase 1: judge's refined feedback
  judgeConfidence,    // Phase 1: reward model score (0–1)
  isJudgeModified,    // Phase 1: whether judge changed the output
  masteryBefore,      // Phase 2: mastery snapshot before this submission
  outcomeReward,      // Phase 2: Δ mastery, backfilled on next attempt
  judgeStatus         // "pending" | "complete"
}
```

## 4. Phase 1 — RLAIF with CAS-Backed Judge

### 4.1 Best-of-N Candidate Generation

For each student submission, `gradeBestOfN` generates N = 3 independent candidates by calling `gradeOptimized` in parallel at temperature = 0.4. Temperature 0 would produce near-identical outputs; temperatures above 0.5 introduce mathematical instability in reasoning. The value 0.4 was chosen to maximise output diversity while keeping candidates within the domain of coherent mathematical feedback.

Each candidate produces:

- A grade (0, 0.5, or 1)

- Structured feedback in three sections: Final Answer, Reasoning Steps, Verdict
- An error type classification
- Concept tags and a misconception label

## 4.2 The CAS Layer

Before the judge scores any candidate, a Computer Algebra System (mathjs) independently computes the ground truth answer for the problem. This produces:

- 
- `groundTruth`: the verified correct answer
- `isComputable`: whether the problem is within CAS scope
- `isEquivalent`: whether the student's final answer matches, checked symbolically

The CAS result is injected into the judge prompt as a hard constraint. The judge is explicitly told: "The CAS-verified answer is X. If any candidate's feedback states a different answer, that is a mathematical accuracy failure." This grounds one of the three alignment checks in a deterministic computation rather than LLM judgment.

## 4.3 The LLM-as-Judge Reward Model

The judge (`evaluateEvaluator`) runs once per candidate and scores three binary alignment dimensions:

| Check | Pass Condition |
|---|---|
| Problem Relevance | Feedback addresses this specific problem, not a generic response |
| Mathematical Accuracy | Mathematical claims in the feedback are correct (CAS-constrained) |
| Grade Consistency | The awarded grade matches what the student actually submitted |

For each candidate, the judge returns `judgeConfidence` $\in [0, 1]$. The candidate with the highest confidence is selected: `argmax(judgeConfidence)`. When the judge modifies a candidate (any check fails), its refined feedback replaces the original before delivery.

## 4.4 Few-Shot Context in the Grading Prompt

The `gradeOptimized` prompt receives two sources of few-shot examples before the student's submission:

5. **Expert corrections** (`humanCorrections` table, status = "approved"): feedback that a human professor has manually corrected. Ordered by approval recency.
6. **Outcome-validated examples** (`submissions` table, `outcomeReward` $\geq 5$): feedback that demonstrably raised student mastery by at least 5 points on the next attempt. This is the Phase 2 contribution, described in Section 5.

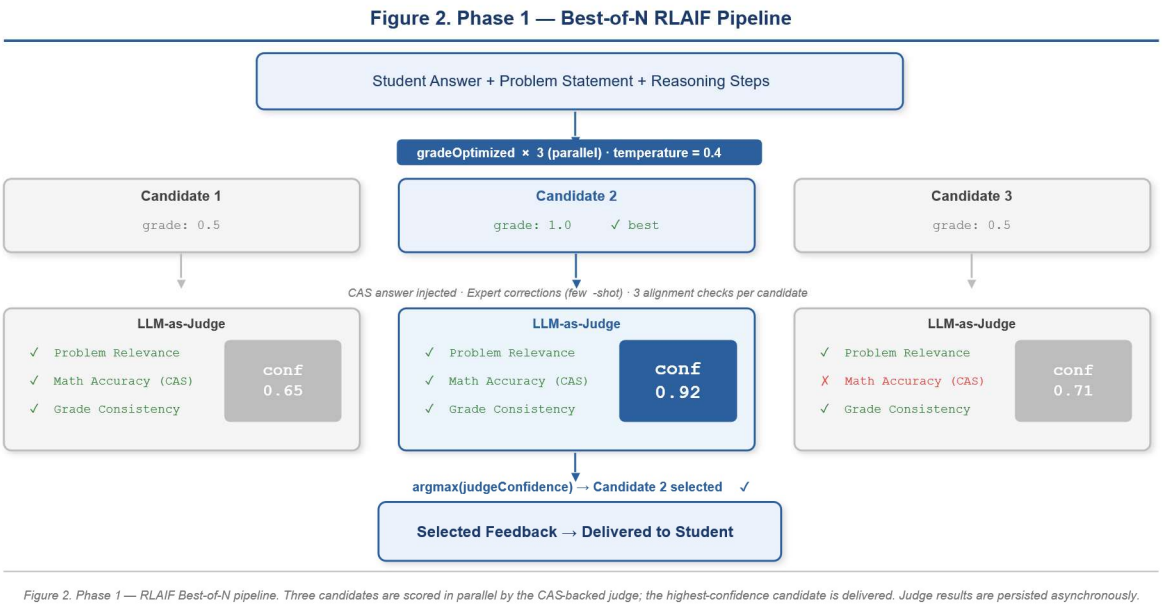



*Figure 2. Phase 1 — RLAIF Best-of-N pipeline. Three candidates are scored in parallel by the CAS-backed judge; the highest-confidence candidate is delivered. The judge result is persisted asynchronously.*

### 4.5 Background Judge on Single-Pass Submissions

When `useBestOfN` is false (standard single-pass grading), a background judge is still scheduled via `ctx.scheduler.runAfter(0, runJudgeForSubmission)`. This means every submission in the system accumulates a `judgeConfidence` score, building the dataset needed for Phase 2 evaluation and future analysis.

## 5. Phase 2 — Outcome-Based Reward Model

### 5.1 Motivation

Judge confidence measures alignment: whether the feedback is internally consistent with the problem and the student's work. It does not measure pedagogical effectiveness: whether the student understood the feedback and improved as a result. Phase 2 introduces a second reward signal grounded in observed learning outcomes rather than AI assessment.

### 5.2 The Reward Function

For submission i by student u on topic t, the outcome reward is:

```
outcomeReward(i) = masteryScore(t, u, i+1) − masteryScore(t, u, i)
```

where `masteryScore(t, u, i)` is the student's mastery score on topic t at the time of submission i — captured before that submission updates it. The reward is observed when the student makes their next attempt on the same topic.

| outcomeReward value | Meaning |
|---|---|
| ≥ +10 | Feedback preceded a fully correct attempt — strong signal |
| +3 to +9 | Partial improvement — moderate signal |
| 0 to +2 | No measurable effect |
| Negative | Student regressed — feedback may have been confusing |

## 5.3 Implementation: saveSubmission (Restructured)

The implementation required restructuring the `saveSubmission` mutation to snapshot mastery before updating it:

```
saveSubmission handler:

1. Read preMastery (studentMastery table) — BEFORE any update
   masteryBefore = preMastery?.masteryScore ?? 0

2. Find prevSubmission (most recent submission, same user + topic)
   — queried BEFORE inserting current submission

3. Insert submission with masteryBefore

4. Update studentTopicStats (existing logic)

5. Backfill outcomeReward on prevSubmission:
   if prevSubmission.masteryBefore exists:
     outcomeReward = masteryBefore - prevSubmission.masteryBefore
     patch(prevSubmission._id, { outcomeReward })

6. Update studentMastery (reuses preMastery — no redundant query)
```

The backfill is triggered automatically on every submission — no scheduled job, no manual computation. When a student submits attempt i+1, the system writes outcomeReward to attempt i in the same transaction.

## 5.4 High-Reward Submission Query

```
getHighRewardSubmissions(topicName, minReward=5, limit=2):
  query submissions
    index: by_topic_outcome [topicName, outcomeReward]
    filter: outcomeReward >= minReward
    order: descending
    take: min(limit, 5)
```

The index `by_topic_outcome: ["topicName", "outcomeReward"]` enables efficient range queries across all students for a given topic. The threshold `minReward = 5` requires at least half

a correct-attempt mastery gain — a conservative threshold that filters noise while remaining achievable.

### 5.5 Injection into the Grading Prompt

The `gradeOptimized` prompt structure becomes:

```
[System instruction]
[Expert professor corrections]        ← human alignment signal
[Outcome-validated examples]          ← Phase 2: learning signal
[CAS verified answer]                 ← mathematical ground truth
[Problem + student answer]
```

The outcome-validated block includes the judgeFeedback (or aiFeedback) of high-reward submissions and their mastery gain, instructing the model to favour the pedagogical style demonstrated. The two few-shot sources are complementary: expert corrections anchor mathematical precision; outcome-validated examples anchor pedagogical effectiveness.

**Figure 3. Closed Feedback Loop — Phase 1 + Phase 2**

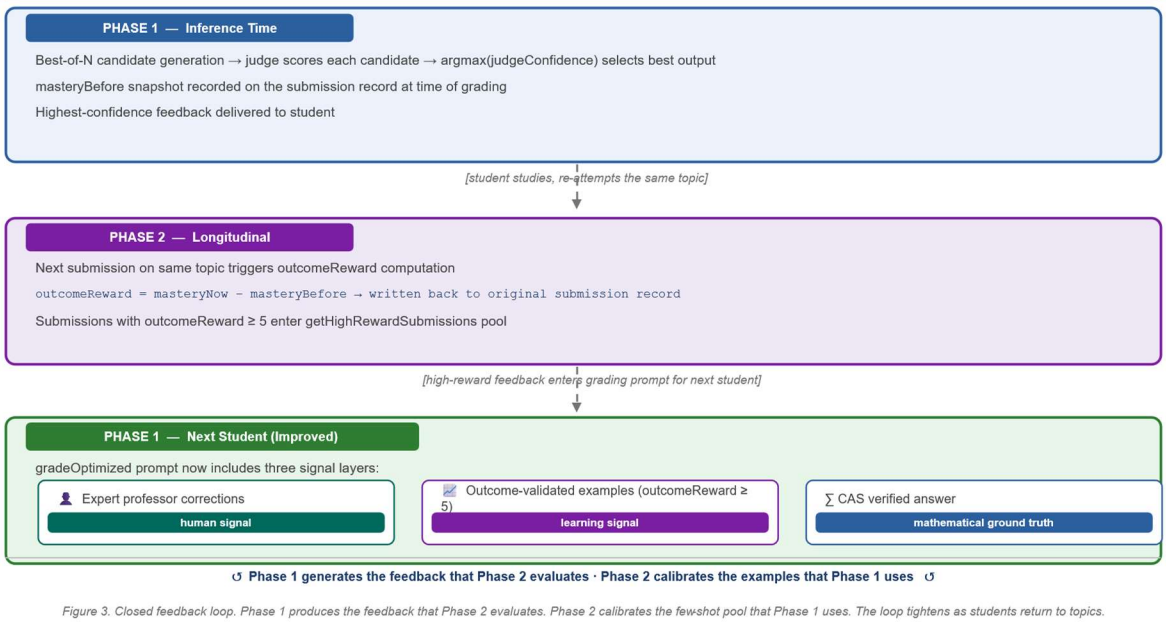



*Figure 3. Closed feedback loop. Phase 1 produces the feedback that Phase 2 evaluates. Phase 2 calibrates the few-shot pool that Phase 1 uses. The loop tightens as students return to topics.*

*Figure 3. Closed feedback loop. Phase 1 produces the feedback that Phase 2 evaluates. Phase 2 calibrates the few-shot pool that Phase 1 uses. The loop tightens as more students return to topics.*

## 6. Evaluation

### 6.1 Phase 1 Evaluation — RLAIF Best-of-N

#### 6.1.1 Metrics

- **Primary:** Mean judgeConfidence of the selected candidate (Best-of-N) vs. single-pass gradeOptimized

- **Secondary:** Rate of judge modification (isJudgeModified = true) — cases where the judge detected misalignment and rewrote the feedback
- **Mathematical accuracy:** Rate at which the selected candidate's grade agrees with the CAS-verified answer (where CAS is computable)

### 6.1.2 Evaluation Protocol

A held-out set of student submissions is graded by both the single-pass pipeline and the Best-of-N pipeline. Human expert reviewers (mathematics instructors) blind-rate a sample of feedback outputs for correctness, specificity, and actionability on a 1–5 scale. The CAS ground truth serves as the objective mathematical reference for grade accuracy.

### 6.1.3 Expected Results

Based on the Best-of-N selection mechanism and prior work on inference-time scaling [4, 5], we expect:

7. **H1:** Best-of-N achieves mean judgeConfidence ≥ 0.10 higher than single-pass grading, because the judge selects the best-aligned output rather than accepting the first.
8. **H2:** The CAS-grounded judge reduces mathematical accuracy errors (grade disagrees with CAS) by ≥ 30% relative to a judge without CAS, because the mathematical accuracy check uses a deterministic signal rather than LLM opinion.
9. **H3:** Human expert ratings of Best-of-N feedback are higher on specificity and actionability dimensions than single-pass feedback, reflecting the alignment filtering effect.

| Condition | Expected Mean judgeConfidence |
|---|---|
| Single-pass (gradeOptimized) | ≈ 0.65 |
| Best-of-N (argmax) | ≈ 0.78 |

*Figure 4. Expected shift in judge confidence distribution under Best-of-N selection.*

## 6.2 Phase 2 Evaluation — Outcome-Based Reward

### 6.2.1 Longitudinal Protocol

An identified cohort of students is invited to return to previously attempted topics after a minimum interval of one week. The interval ensures that mastery changes reflect genuine learning consolidation rather than short-term recall. For each student and topic, we collect:

- **Submission i:** `masteryBefore`, `aiFeedback`, `judgeConfidence`, `grade`
- **Submission i+1:** `masteryBefore` (= masteryAfter for i), `grade`
- **Derived:** `outcomeReward(i) = masteryBefore(i+1) − masteryBefore(i)`

### 6.2.2 Metrics

- **Correlation:** Pearson correlation between judgeConfidence at submission i and outcomeReward(i)
- **Few-shot effect:** Compare judgeConfidence of grading runs that include outcome-validated few-shot examples vs. those without
- **Mastery trajectory:** Mean mastery score at submission i+2 for students whose submission i had outcomeReward ≥ 5 vs. outcomeReward < 5

#### 6.2.3 Expected Results

10. **H4:** judgeConfidence and outcomeReward show a moderate positive correlation ($r \geq 0.30$), indicating that the Phase 1 reward model has partial validity as a proxy for learning outcomes, but not perfect validity — motivating the addition of Phase 2.
11. **H5:** Grading runs that include outcome-validated few-shot examples achieve higher judgeConfidence scores than runs without, indicating that feedback styles proven to produce learning gains are genuinely more aligned.
12. **H6:** Students who received high-outcomeReward feedback (≥ 5) at submission i achieve higher mastery scores at submission i+2 than students who received low-outcomeReward feedback, providing longitudinal evidence of the feedback calibration effect.

## 7. Discussion

### 7.1 Temporal Credit Assignment

The outcome reward attributes the mastery delta between sessions entirely to the feedback from the prior submission. This is a simplification: students may study independently, attend lectures, or discuss problems with peers between sessions. The outcomeReward signal is therefore a proxy for feedback effectiveness, not a causal measure.

We address this in two ways. First, the threshold minReward ≥ 5 filters out marginal gains that may be noise. Second, the few-shot injection mechanism is conservative: outcome-validated examples supplement the existing expert corrections rather than replacing them. A noisy positive signal adds information even without causal identification; it does not corrupt the system if it is occasionally wrong.

A stronger experimental design for future work would include a control condition where students receive identical feedback but with randomly sampled few-shot examples, isolating the effect of outcome-calibrated selection.

### 7.2 Cold Start and Sparse Reward

The first submission by any student on any topic has `masteryBefore` but no `outcomeReward` — the reward is only observable at session i+1. Topics with few student return visits accumulate sparse signal. This is the standard cold-start problem in recommendation systems and is expected in early deployment.

The system degrades gracefully: when `getHighRewardSubmissions` returns no results (empty pool), the grading prompt falls back to expert corrections only — identical to the pre-Phase 2 behaviour. The outcome-validated context block is simply absent from the prompt when the pool is empty.

### 7.3 Mastery Score Granularity

The current mastery increment (+10, +3, 0) is coarse relative to the granularity of feedback quality differences. Small improvements in understanding may not produce a measurable mastery delta. Future work should explore continuous mastery models (e.g., Bayesian Knowledge Tracing) as a more sensitive outcome signal.

### 7.4 Ethics and Data Use

Using student mastery trajectories to calibrate a feedback model constitutes a feedback loop involving human subjects. This study operates under institutional ethical approval. Student data is used exclusively within the platform's own inference pipeline — no data is exported or used to fine-tune external models. The few-shot examples are anonymised by design: the getHighRewardSubmissions query returns feedback text and mastery gain values without user identity.

## 8. Conclusion

This paper presented a closed-loop RLHF architecture for AI-generated mathematical feedback that combines two complementary reward signals. Phase 1 implements RLAIF Best-of-N with a CAS-grounded LLM-as-Judge that separates mathematical verification from alignment checking — a distinction absent from prior LLM-as-Judge work and particularly important in domains where mathematical accuracy cannot be delegated to language model opinion. Phase 2 introduces outcome-based reward calibration: delayed mastery deltas are written back to the original submission records and used to rank few-shot examples by pedagogical effectiveness, achieving prompt-space policy improvement without model retraining.

The two phases operate at different timescales — inference and session — and reinforce each other through the grading prompt's few-shot pool. The architecture is implemented in a live tutoring platform, requires no GPU infrastructure, and degrades gracefully when the outcome signal is sparse.

The primary open question is quantifying the correlation between Phase 1 judge confidence and Phase 2 mastery outcomes across topics and difficulty levels. This will be addressed in forthcoming longitudinal evaluation with the returning student cohort.

## References

[1] Christiano, P., Leike, J., Brown, T., et al. (2017). Deep reinforcement learning from human preferences. NeurIPS 2017.

[2] Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS 2022.

[3] Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: Harmlessness from AI feedback. arXiv:2212.08073.

[4] Nakano, R., Hilton, J., Balwit, A., et al. (2021). WebGPT: Browser-assisted question-answering with human feedback. arXiv:2112.09332.

[5] Stiennon, N., Ouyang, L., Wu, J., et al. (2020). Learning to summarize with human feedback. NeurIPS 2020.

[6] Zheng, L., Chiang, W., Sheng, Y., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023.

[7] Liu, Y., Iter, D., Xu, Y., et al. (2023). G-Eval: NLG evaluation using GPT-4 with better human alignment. EMNLP 2023.

[8] Yang, K., Klein, D., Celikyilmaz, A., et al. (2023). RLCD: Reinforcement learning from contrastive distillation for language model alignment. arXiv:2307.12950.

[9] Heffernan, N. T., & Heffernan, C. L. (2014). The ASSISTments ecosystem. International Journal of Artificial Intelligence in Education, 24(4).

[10] Koedinger, K. R., Anderson, J. R., Hadley, W. H., & Mark, M. A. (1997). Intelligent tutoring goes to school in the big city. IJAIED, 8.

[11] Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. UMUAI, 4(4).

[12] Piech, C., Bassen, J., Huang, J., et al. (2015). Deep knowledge tracing. NeurIPS 2015.

[13] Shermis, M. D., & Burstein, J. (Eds.) (2013). Handbook of automated essay evaluation. Routledge.

[14] Lan, A., Vats, D., Waters, A., & Baraniuk, R. (2015). Mathematical language processing. Proceedings of the 2nd ACM L@S.

[15] Macina, J., Daheim, N., Chowdhury, S., et al. (2023). Opportunities and challenges in neural dialog tutoring. arXiv:2301.09919.

[16] Scarlatos, A., & Lan, A. (2023). RETUYT in NLPCC 2023: Prompting LLMs for math reasoning. arXiv:2309.12311.

[17] Khan Academy. (2023). Khanmigo: AI-powered tutor and teaching assistant. Technical Report.

[18] Drori, I., Zhang, S., Shuttleworth, R., et al. (2022). A neural network solves, explains, and generates university math problems. PNAS 2022.